Relazione esercizio 51 TPSI

Marco Schiavello 4°CI

28/05/2022

INDICE

INDICE 2

IDE utilizzato 3

U.M.L. 3

Annotazione MachineOwner 3

ShutdownHook MainShutdownHook 4

Classe Consumatore 5

Classe Macchina 7

Classe Servitore 10

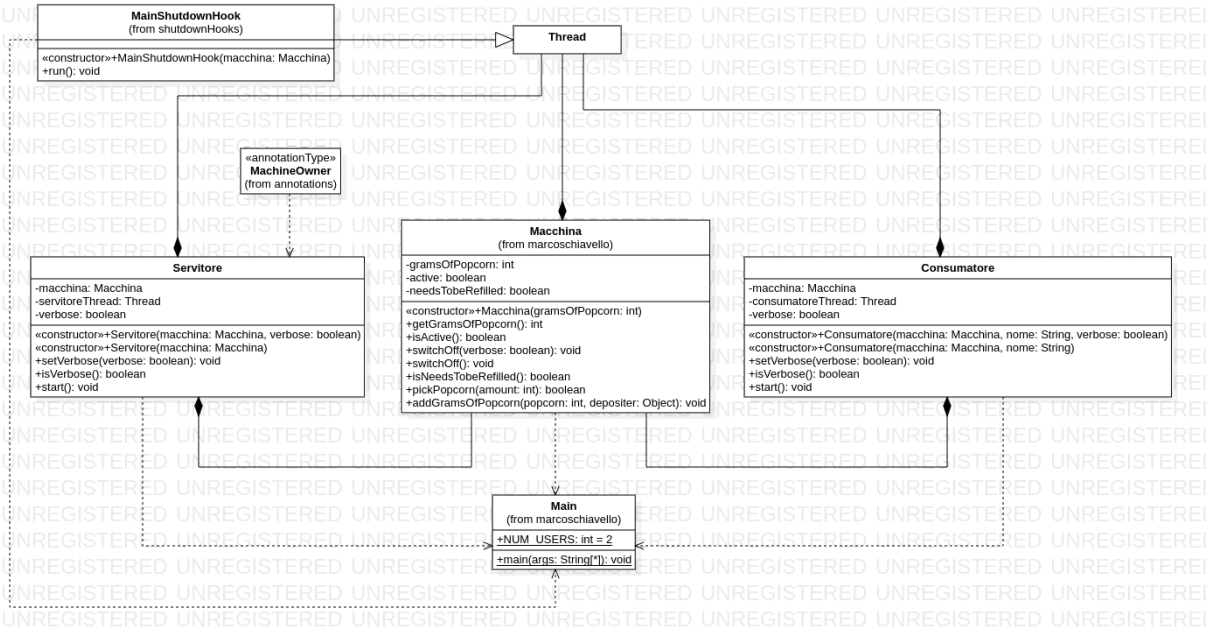
Classe Main 12

Spiegazione generale del progetto 13

IDE utilizzato

Immagini non utilizzare Intellij, imbarazzante, comunque ho usato Intellij.

U.M.L.



Annotazione MachineOwner

Annotazione che indica che l’istanze di una classe annotate da questa annotazione possono aggiungere popcorn alla macchina

```
/*******************************************************
*                      MachineOwner                    *
*******************************************************/

package com.marcoschiavello.annotations;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/**
```

```
 * Annotazione che indica una un istaza di una classe annotata con
questa annotazione puo aggiungere popcorn alla macchina
 *
 * @author Marco Schiavello
 */
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface MachineOwner { }
```

ShutdownHook MainShutdownHook

Shutdown hook per gestire la conclusione della applicazione, prende una macchina in modo da terminare con il metodo prestabilito in modo gracefully

```
/*******************************************************
*                     MainShutdownHook                 *
*******************************************************/

package com.marcoschiavello.shutdownHooks;

import com.marcoschiavello.Macchina;

/**
 * Shutdown hook per gestire la conclusione della applicazione,
prende una macchian in modo da teminarla con il metodo prestabilito
in modo gracefully
 *
 * @author Marco Shciavello
 */
public class MainShutdownHook extends Thread{
    private Macchina macchina;

    public MainShutdownHook(Macchina macchina) { this.macchina =
macchina; }

    @Override
    public void run() { this.macchina.switchOff(); }

}
```

Classe Consumatore

Classe che indica un consumatore che prende i popcorn dalla macchina ogni secondo e sincronizza la stampa e presa dei popcorn sull'oggetto della macchina se la richiesta di popcorn e più grande della disponibilità della macchina allora verrà chiamato wait() che lascerà il lock all servitore che riempirà la macchina

```
/*******************************************************
*                     Consumatore                      *
*******************************************************/

package com.marcoschiavello;

/**
 * Classe che indica un consumatore che prende i popcorn dalla
machcina ogni secondo e sincrozinna la stampa e presa dei popcorn
sull'oggetto della macchina
 * se la richiesta di popcorn e piu grande della disponibilita della
macchina allora verra chiamato wait() che lasciera il lock
all'servitore che riempirà la machina
 *
 * @author Marco Schiavello
 */
public class Consumatore {
    /**
     * macchiina a cui fa riferimento il consumatore
     */
    private Macchina macchina;
    /**
     * thread che esegue le azzioni del consumatore in modo
concorente
     */
    private Thread consumatoreThread;
    /**
     * flag che dice se sampre messaggi sullo stato dell'eseguzione
     */
    private boolean verbose;

    /**
```

```
     * costruttore completo che crea un consumatore
     *
     * @param macchina macchiina a cui fa riferimento il consumatore
     * @param nome nome del thread del consumatore
     * @param verbose flag che dice se sampre messaggi sullo stato
dell'eseguzione
     */
    public Consumatore(Macchina macchina, String nome, boolean
verbose) {
        this.macchina = macchina;
        this.verbose = verbose;
        this.consumatoreThread = new Thread(() -> {
            while(macchina.isActive()) {
                synchronized(macchina) {
                    while(!macchina.pickPopcorn(100)) {
                        try { macchina.wait(); }
catch(InterruptedException e) { }
                    }

                    if(this.verbose)
                        System.out.println(nome + " ha preso 100
grammi di popcorn, rimanenti = " + macchina.getGramsOfPopcorn() +
"g");
                }

                try { consumatoreThread.sleep(1000); }
catch(InterruptedException e) { }
            }
        }, nome);

    }

    /**
     * costruttore parziale che crea un consumatore
     *
     * @param macchina macchiina a cui fa riferimento il consumatore
     * @param nome nome del thread del consumatore
     */
    public Consumatore(Macchina macchina, String nome) {
this(macchina, nome, true); }
```

```
    /**
     * setter di verbose
     *
     * @return valore di verbose
     */
    public boolean isVerbose() {return this.verbose; }

    /**
     * getter di verbose
     */
    public void setVerbose(boolean verbose) { this.verbose =
verbose; }

    /**
     * matodo wrapper di {@link Thread#start()}
     */
    public void start() { this.consumatoreThread.start(); }
}
```

Classe Macchina

Classe che identifica una macchina di popcorn che sarà usata da N consumatori e gestita e riempita da 1 a N servitori

```
/*******************************************************
*                     Macchina                         *
*******************************************************/
package com.marcoschiavello;

import com.marcoschiavello.annotations.MachineOwner;

/**
 * Classe che identifica una macchina di popcorn che sarà usata da N
consumatori e gestita e riempita da 1 a N servitori
 *
 * @author Marco Shciavello
 */
public class Macchina {
    /**
```

```
     * grammi di popcorn dentro la macchina
     */
    private int gramsOfPopcorn;
    /**
     * flag che indica se la macchina è attiva o meno
     */
    private boolean active;
    /**
     * flag che indica se la macchina deve essere riempita da un
servitore
     */
    private boolean needsTobeRefilled;

    /**
     * costruttore completo che crea una macchiana di popcorn
     *
     * @param gramsOfPopcorn grammi di popcorn iniziali
     */
    public Macchina(int gramsOfPopcorn) {
        this.gramsOfPopcorn = gramsOfPopcorn;
        this.active = true;
        this.needsTobeRefilled = false;
    }

    /**
     * getter di gramsOfPopcorn sincronizzato sull'oggetto macchina
corrente
     */
    public synchronized int getGramsOfPopcorn() { return
this.gramsOfPopcorn; }

    /**
     * getter del flag active
     */
    public boolean isActive() { return this.active; }

    /**
     * metodo che mette a false l'attributo active
     */
    public void switchOff(boolean verbose) {
        this.active = false;
```

```
        if(verbose)
            System.out.println("La macchina dei popcorn si sta
spegnendo");
    }

    /**
     * scorciatoia del metodo che mette a false l'attributo active
     */
    public void switchOff() { this.switchOff(true); }

    /**
     * getter del flag needsTobeRefilled
     */
    public boolean isNeedsTobeRefilled() { return
this.needsTobeRefilled; }

    /**
     * metodo sincronizzato sull'oggetto macchina corrente che
preleva una quantita N di popcorn dalla macchina dove N è un numero
preso come parametro
     *
     * @param amount quantita di popcorn da prendere
     */
    public boolean pickPopcorn(int amount) {
        this.needsTobeRefilled = amount > this.gramsOfPopcorn;

        this.gramsOfPopcorn -= (amount * (this.needsTobeRefilled ? 0
: 1));
        return !this.needsTobeRefilled;
    }

    /**
     * metodo sincronizzato sull'oggetto macchina corrente che
aggiunge una quantita N di popcorn nella macchina dove N è un numero
preso come parametro
     * l'azione di aggiunta è controllata e puo essere fatto solo da
istanze di classe che sono annotate con l'annotazione {@link
MachineOwner}
     *
     * @param popcorn quantita di popcorn da aggiugere
```

```
     * @param depositer istanza dell'oggetto che vuole aggiungere i
popcorn
     */
    public void addGramsOfPopcorn(int popcorn, Object depositer) {

if(depositer.getClass().isAnnotationPresent(MachineOwner.class))
            this.gramsOfPopcorn += popcorn;
    }
}
```

Classe Servitore

Classe che identifica un servitore che controlla ogni 100 millisecondi se la macchina deve essere riempite se si allora la riempirà con il suddetto metodo

```
/*******************************************************
*                          Servitore                   *
*******************************************************/

package com.marcoschiavello;

import com.marcoschiavello.annotations.MachineOwner;

/**
 * Classe che identifica un servitore che controlla ogni 100
millisecondi se la macchian deve essere riempite se si allora la
riempira con il suddetto metodo
 *
 * @author Marco Schiavello
 */
@MachineOwner
public class Servitore {
    /**
     * macchiina a cui fa riferimento il servitore
     */
    private Macchina macchina;
```

```
    /**
     * thread che esegue le azzioni del servitore in modo concorente
     */
    private Thread servitoreThread;
    /**
     * flag che dice se sampre messaggi sullo stato dell'eseguzione
     */
    private boolean verbose;

    /**
     * costruttore completo che crea un servitore
     *
     * @param macchina macchiina a cui fa riferimento il servitore
     * @param verbose flag che dice se sampre messaggi sullo stato
dell'eseguzione
     */
    public Servitore(Macchina macchina, boolean verbose) {
        this.macchina = macchina;
        this.verbose = verbose;
        this.servitoreThread = new Thread(() -> {
            while(macchina.isActive()) {
                if(macchina.isNeedsTobeRefilled()) {
                    synchronized (macchina) {
                        this.macchina.addGramsOfPopcorn(1000, this);
                        if(this.verbose)
                            System.out.println("Il servitore sta
riempiendo la macchina aspetta ...");
                        try { servitoreThread.sleep(2000); }
catch(InterruptedException e) { }
                        macchina.notifyAll();
                    }
                }

                try { servitoreThread.sleep(100); }
catch(InterruptedException e) { }
            }
        });

        this.servitoreThread.setDaemon(true);
    }
```

```
    /**
     * costruttore parziale che crea un servitore
     *
     * @param macchina macchiina a cui fa riferimento il servitore
     */
    public Servitore(Macchina macchina) { this(macchina, true); }

    /**
     * setter di verbose
     *
     * @return valore di verbose
     */
    public boolean isVerbose() {return this.verbose; }

    /**
     * getter di verbose
     */
    public void setVerbose(boolean verbose) { this.verbose =
verbose; }

    /**
     * matodo wrapper di {@link Thread#start()}
     */
    public void start() { this.servitoreThread.start(); }
}
```

Classe Main

La classe main in questo programma si occupa di gestire la creazione e avvio dei thread consumatori(che sono messi in un arrayList) e dei/dell servitore/i.
Alla fine il main aspetta 30s prima di spegnere la macchina e concludere

```
/*******************************************************
*                          Main                        *
*******************************************************/

package com.marcoschiavello;

import com.marcoschiavello.shutdownHooks.MainShutdownHook;

import java.util.ArrayList;
import java.util.List;
```

```
public class Main {
    public static int NUM_USERS = 2;

    public static void main(String[] args) throws
InterruptedException {
        Macchina macchina = new Macchina(1000); // crea una macchina
con un valore di popcorn di base di 1000g
        Servitore servitore = new Servitore(macchina); // crea un
servitore che continuera a riempire la macchina di 1000g ogni volta
che i popcorn dentro la macchina non sono sufficenti per soddisfare
una richiesta
        List<Consumatore> consumatori = new ArrayList<>(); // lista
di consumatori

        servitore.start(); // parte il servitore
        for(int i = 0; i < NUM_USERS; i++) {
            consumatori.add(new Consumatore(macchina, "consumatore-"
+ i)); // crea un servitore e lo aggiunge alla lista
            consumatori.get(i).start(); // fa partire il servitore
        }

        Thread.currentThread().sleep(30000); // fa continuare il
programma per 30 secondi
        macchina.switchOff(false); //spegne la macchina dopo i 30
secondi
        Runtime.getRuntime().addShutdownHook(new
MainShutdownHook(macchina)); // aggiunge uno shutdown hook per la
gestione della chiussura del programma
    }
}
```

Spiegazione generale del progetto

Questo programma aveva come scopo di mettere in risalto i metodi wait(), notify(), notifyAll() che permettono una gestione più accurata dei thread che previene certe situazioni di deadlock causate da un circolo vizioso di attese dei processi gli uni con gli altri.
In questo programma è stato molto utile l'utilizzo di wait() mette il thread in waiting ma rilasciando il lock che deteneva facendo così subentrare altri thread che permettono l'evoluzione del thread in waiting, e poi è stato utile anche l'utilizzo della funzione notifyall() che risveglia tutti i trad in waiting su un determinato lock, in questo caso e usato per

avvertire i consumatori che il servitore ha riempito la macchine e posso dunque continuare a prendere popcorn.